Wilo-Pumpe Teil 2: Core 1 richtig nutzen

MicroPython verwaltet beide Kerne — so laeuft eigener C-Code trotzdem

make Magazin · Projekt-Doku

April 2026 `github.com/gerontec/wilo_pwm`

Der RP2040 hat zwei Kerne — aber MicroPython v1.28 nutzt beide. Wer Core 1 direkt startet, crasht das System. Dieser Artikel zeigt, wie man trotzdem eigenen C-Code auf Core 1 bekommt: ueber MicroPythons Thread-System, mit 26 Millionen Polling-Iterationen pro Sekunde.

---

1 Die Fehlannnahme: Core 1 liegt brach

Eine verbreitete Annahme beim RP2040 unter MicroPython lautet: *"Core 0 laeuft MicroPython, Core 1 ist frei."* Das stimmt nicht.

MicroPython v1.28 fuer den rp2-Port kompiliert mit `MICROPY_PY_THREAD=1` — Threading ist standardmaessig aktiv. Die Datei `mpthreadport.c` registriert einen eigenen `core1_entry_wrapper` und ruft bei Bedarf `multicore_launch_core1_with_stack()` auf.

Der Fehler liegt nahe: Ein eigenes `multicore_launch_core1(meine_funktion)` direkt aus C aufzurufen kollidiert mit dieser Infrastruktur — mit vorhersehbarem Ergebnis:

Listing 1: Typisches Fehlerbild beim direkten Core-1-Start

```
# Pico bootet, USB erscheint kurz...
[usb 1-1: device descriptor read/64, error -71]
# ...und verschwindet wieder. Kein ttyACM0, kein WebREPL.
```

Der Crash tritt auf bevor MicroPython die Python-VM initialisiert hat, weil Core 1 sofort in eine enge Polling-Schleife springt und dabei den USB-Stack stoert.

2 Die richtige Schnittstelle: mp_thread_create

MicroPython stellt eine offizielle C-API bereit, um Code auf Core 1 zu starten: `mp_thread_create()`. Diese Funktion leitet den Aufruf durch `core1_entry_wrapper` — sie ist kompatibel mit MicroPythons interner Multicore-Verwaltung.

Listing 2: C-Modul: Core 1 sicher starten

```c
#include "py/runtime.h"
#include "py/mpthread.h"
#include "pwmfb_core1_shared.h"
#include "pwmfb_core1_task.h"

static mp_obj_t mp_c1_start(void) {
```

```c
    static bool started = false;
    if (!started) {
        size_t stack_size = 1024;
        mp_thread_create(pwmfb_core1_task, NULL, &stack_size);
        started = true;
    }
    return mp_const_none;
}
```

Der `static bool`-Guard verhindert, dass `mp_thread_create` bei jedem Python-`import` erneut aufgerufen wird — wichtig, weil MicroPythons Soft-Reset das Modul neu initialisiert.

3 Der Task: Polling mit Batch und Yield

Ein weiteres Detail: MicroPythons Thread-System erwartet, dass der Thread gelegentlich Zeit abgibt. Eine nackte `while(1)`-Schleife ohne jeden Yield startet, crasht dann aber den Interpreter. Die Loesung ist ein Batch-Ansatz:

Listing 3: pwmfb_core1_task.c — Batch-Polling mit sleep_us

```c
#define POLL_BATCH  10000u   // Iterationen pro Runde

void *pwmfb_core1_task(void *arg) {
    (void)arg;
    uint32_t last_t   = TIMER_TIMELR;
    uint8_t  last_lvl = (SIO_GPIO_IN & PIN_MASK) ? 1 : 0;

    while (1) {
        for (uint32_t i = 0; i < POLL_BATCH; i++) {
            g_pwmfb.poll_count++;
            uint8_t lvl = (SIO_GPIO_IN & PIN_MASK) ? 1 : 0;
            if (lvl != last_lvl) {
                uint32_t now  = TIMER_TIMELR;
                uint32_t diff = now - last_t;
                if (diff >= DEBOUNCE_US) {
                    if (lvl == 1) g_pwmfb.low_us  = diff;
                    else          g_pwmfb.high_us = diff;
                    g_pwmfb.last_upd = now;
                    g_pwmfb.ready    = 1;
                    g_pwmfb.edge_count++;
                } else {
                    g_pwmfb.debounce_drop++;
                }
                last_t   = now;
                last_lvl = lvl;
            }
        }
        sleep_us(1);   // kurz yielden -- haelt MicroPython stabil
    }
    return NULL;
}
```

10 000 Polling-Iterationen dauern auf dem RP2040 bei 125 MHz etwa 80 us, dann folgt 1 us Pause. Das ergibt in der Praxis **26 Millionen Polls pro Sekunde** bei gleichzeitig stabilem MicroPython-Betrieb.

4 Shared Memory: Keine Locks noetig

Core 1 schreibt, Core 0 liest. Die Struct liegt im statischen `.data`-Segment der Firmware:

Listing 4: pwmfb_core1_shared.h

```
typedef struct {
    volatile uint32_t high_us;        // HIGH-Dauer letzter Flanke
    volatile uint32_t low_us;         // LOW-Dauer
    volatile uint32_t last_upd;       // Zeitstempel (TIMER_TIMELR)
    volatile uint32_t poll_count;     // Gesamt-Iterationen
    volatile uint32_t edge_count;     // Erkannte Flanken
    volatile uint32_t debounce_drop;  // Verworfene Flanken
    volatile uint32_t debounce_us;    // aktueller Schwellwert (adaptiv)
    volatile uint8_t  ready;
    volatile uint8_t  _pad[3];
} pwmfb_shared_t;

extern pwmfb_shared_t g_pwmfb;
```

Auf dem RP2040 sind 32-Bit-Schreibzugriffe auf den SRAM atomar (Single-Cycle-SRAM, 32-Bit-Bus, ausgerichtete Adressen). Ein Mutex ist fuer diese Lesemuster nicht noetig.

5 Adaptiver Debounce: automatisch fuer jede Frequenz

Ein fester Debounce-Schwellwert funktioniert nur in einem engen Frequenzbereich. Bei 75 Hz (Periode 13,3 ms) sind 50 us harmlos — bei 10 kHz (Periode 100 us) wuerden bereits 50 % aller Flanken verworfen, bei 1 MHz (Periode 1 us) jede einzelne.

Die Loesung: Core 1 berechnet den Schwellwert nach jeder erfolgreichen Messung neu — 5 % der gemessenen Periode, geklemmt zwischen 2 us und 5000 us:

Listing 5: Adaptive Debounce nach jeder Flanke

```
// Nach erfolgreicher Messung (innerhalb des Polling-Batch):
uint32_t period  = g_pwmfb.high_us + g_pwmfb.low_us;
uint32_t new_deb = period / 20;   // 5% der Periode
if (new_deb < DEBOUNCE_US_MIN) new_deb = DEBOUNCE_US_MIN; // min 2 us
if (new_deb > DEBOUNCE_US_MAX) new_deb = DEBOUNCE_US_MAX; // max 5000 us
g_pwmfb.debounce_us = new_deb;
deb = new_deb;   // sofort im laufenden Batch wirksam
```

Der Schwellwert passt sich bei jedem Frequenzwechsel automatisch an — kein Neustart, kein Konfigurationsparameter. Der aktuelle Wert ist per `get_health()` ablesbar und wird per MQTT als `C1_deb_us` publiziert:

Signal	Periode	Debounce (adaptiv)
75 Hz	13 333 us	666 us
1 kHz	1 000 us	50 us
10 kHz	100 us	5 us
500 kHz	2 us	2 us (Minimum)

6 Health-Check: Ist Core 1 noch am Leben?

Das Python-Interface gibt drei Kenngroessen zurueck:

Listing 6: Health-Check aus MicroPython

```
import pwmfb_core1
pwmfb_core1.start()

# get_health() -> (poll_count, edge_count, debounce_drop, ready,
    debounce_us)
poll, edges, drops, ready, deb = pwmfb_core1.get_health()
print(f"Polls/s: ~{poll // uptime_s:,}")
print(f"Flanken total: {edges}")
print(f"Debounce-Verwuerfe: {drops}")
```

Typischer Messwert nach 14 Sekunden Laufzeit:

Kenngroesse	Wert
poll_count	46 825 388
edge_count	2 777 (bei 75 Hz ≈ 150 Flanken/s)
debounce_drop	0
debounce_us	666 us (adaptiv bei 75 Hz)
ready	1

`poll_count` wachst kontinuierlich — bleibt er stehen, ist Core 1 haengengeblieben. MQTT traegt beide Werte bei jedem 5-Sekunden-Publish aus.

7 Frequenzmessung: 75 Hz garantiert

Die Wilo iPWM-Pumpe sendet exakt 75 Hz. Core 1 misst HIGH- und LOW-Dauer jeder Periode direkt per Registerlesezugriff auf `TIMER_TIMELR` (1-us-Aufloesung). Die Python-Seite berechnet daraus Frequenz und Duty-Cycle:

Listing 7: Frequenzberechnung im Python-Wrapper

```
high_us, low_us, age_us = pwmfb_core1.get_raw()
T    = high_us + low_us       # Periodenlaenge in us
freq = 1_000_000.0 / T        # Frequenz in Hz
duty = high_us / T * 100.0    # Duty-Cycle in Prozent
```

Gemessene Werte im Betrieb: `75.0 Hz`, Duty-Cycle je nach Betriebspunkt 29–42 %.

8 Build: USER_C_MODULES statt natmod

Im Gegensatz zum natmod-Ansatz aus Teil 1 wird das Core-1-Modul als `USER_C_MODULES` in die Firmware gebaut — es benoetigt Zugriff auf MicroPythons interne Thread-API, die im natmod-Kontext nicht verfuegbar ist.

Listing 8: Firmware bauen

```
cd ~/micropython/ports/rp2
git checkout v1.28.0
git submodule update --init

make BOARD=RPI_PICO_W \
     USER_C_MODULES=/home/gh/python/pwmfb_core1/micropython.cmake \
     -j$(nproc)
# -> build-RPI_PICO_W/firmware.uf2
```

Das MicroPython-Dateisystem (main.py, .mpy-Dateien) bleibt unveraendert. Nach dem Flash einfach per WebREPL oder mpremote neu deployen.

Ansatz	**Max. Freq.**	**Besonderheit**
Python IRQ (pure)	∼1–2 kHz	kein Build
C-Natmod (.mpy)	∼10 kHz	kein Build, portabel
Core 1 via Thread	∼500 kHz	Custom Firmware noetig
PIO (Assembler)	∼62 MHz	Custom Firmware + ASM

Fuer 75 Hz ist jeder Ansatz ausreichend. Der Wert liegt im Debugging: `poll_count` und `edge_count` per MQTT machen Core-1-Ausfall sofort sichtbar.

Code, Firmware & Deploy-Script: https://github.com/gerontec/wilo_pwm